MODULE 2 · SESSION 1

Python for AI - The API Middleware Layer

Session 1 — Foundations of AI Backend Engineering

Instructor	Duration
Shrey · AI Solution Engineer	2-Hour Classroom Session

Module

Applied Generative AI Engineering

YOUR INSTRUCTOR

Meet Shrey

AI Solution Engineer specialising in designing, training, and deploying machine learning models - and building production AI systems that integrate LLM APIs, vector databases, and backend services.

Technical Expertise

- Generative AI & Large Language Models (LLMs)
- Retrieval-Augmented Generation (RAG) architectures
- Multi-agent and agentic AI workflows
- NLP, Computer Vision & ML model development
- AI backend system architecture & vector databases

Industry Domains

- Supply chain & trade intelligence platforms
- Medical & healthcare AI systems
- Recruitment & hiring automation
- Conversational AI systems & chatbots

COURSE ROADMAP

Your Learning Journey

This course is structured as a progressive build - each module layers on the last, culminating in production-ready AI systems. **This module focuses on the backend middleware layer** that connects applications to AI models.

AI Foundations

Core concepts, LLMs, and the GenAI landscape

Python Middleware ← You Are Here

Backend architecture, APIs, environments, version control

RAG Systems

Vector databases, embeddings, retrieval pipelines

Agentic AI Systems

Multi-agent workflows, tool use, orchestration

Deployment

Containerisation, CI/CD, production AI services

SESSION 1 · OBJECTIVES

What You Will Learn Today

By the end of this session, you will have a working mental model of how AI applications are structured and the tools every AI backend engineer uses daily.

1

AI App Architecture

Understand how modern AI products are structured end-to-end

2

Python as Middleware

See how Python acts as the backend glue between users and AI models

3

APIs Explained

Define what APIs are and why they are the connective tissue of AI systems

4

Virtual Environments

Create isolated Python environments for conflict-free development

5

Git & GitHub

Apply version control fundamentals to track and share code

6

Secure Credentials

Manage API keys safely using environment variables and `.env` files

ARCHITECTURE

How Modern AI Products Work

Every AI product you interact with — ChatGPT, Gemini, Copilot - follows a layered architecture. This **5-layer stack** details how user requests are processed and responses are delivered.

User

Browser / Mobile App

Frontend Application

React / Web / Mobile UI

Backend API · Flask / FastAPI / Django

THIS MODULE

Middleware layer - processes requests, calls AI services, returns responses

AI APIs / ML Models

OpenAI, Gemini, Claude, or custom ML models

Data Systems

Vector Database / Database / Knowledge Base

This Python backend layer is the middleware we will build and work with in this module.

BACKEND ENGINEERING

Responsibilities of the Backend Layer

The backend is not just a pass-through. It performs critical orchestration work that keeps AI applications reliable, secure, and scalable.

Process User Requests

Parse, validate, and route incoming requests from the frontend

Interact with APIs

Construct payloads and call external AI services programmatically

Communicate with ML Models

Send inputs to models and interpret structured outputs

Manage Authentication

Handle API keys, tokens, and access control securely

Return Responses

Format and deliver results back to the frontend or client

CORE CONCEPT

What is an API?

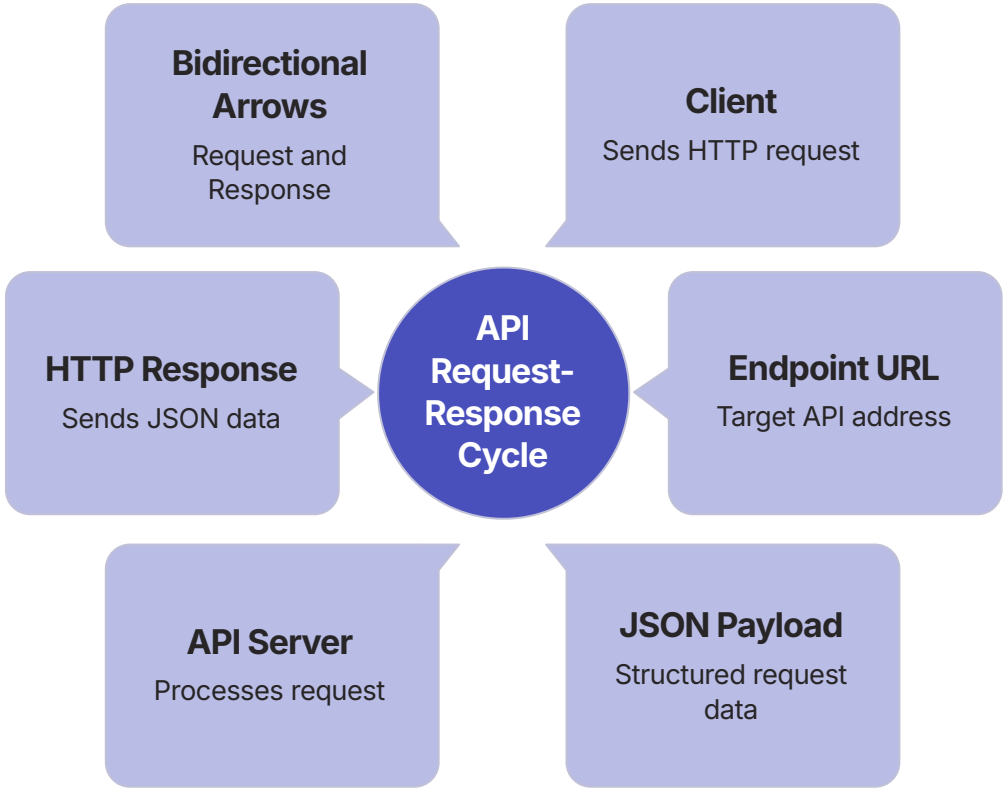
Definition

Application Programming Interface (API) — a defined contract that allows two software systems to communicate with each other over a network.

APIs expose a set of **endpoints** (URLs). You send a structured **request** (with data), and receive a structured **response** (usually JSON).

Key Properties

- Language-agnostic — any system can call any API
- Standardised over HTTP/HTTPS
- Protected by authentication (API keys, tokens)
- Returns predictable, structured data



API ANALOGY

Think of an API Like a Restaurant

You don't walk into the kitchen and cook your own food. You tell the waiter what you want, and the kitchen handles the rest. APIs work exactly the same way — they abstract complexity behind a clean interface.

1. **Customer**
   Places an order from the menu
2. **Waiter (API)**
   Takes the request to the kitchen
3. **Kitchen (Server)**
   Processes the request, prepares response
4. **Waiter (API)**
   Delivers the result back
5. **Customer**
   Receives exactly what was ordered

In AI engineering: **Your Python code = Customer** · **OpenAI API = Waiter** · **GPT-4 Model = Kitchen**

WHY PYTHON?

Why Python is the Language of AI Engineering

Python dominates AI and ML engineering because of its readable syntax, massive ecosystem, and first-class support from every major AI provider. Every leading AI framework — from TensorFlow to LangChain — is Python-first.

Key Advantages

- **Simple syntax** — reads almost like plain English, low cognitive overhead
- **Vast ecosystem** — 500,000+ packages on PyPI
- **ML & AI libraries** — NumPy, PyTorch, TensorFlow, scikit-learn, LangChain
- **API integration** — `requests`, `httpx`, and SDK libraries for every major AI provider
- **Rapid prototyping** — go from idea to working prototype in hours

Popular AI Backend Frameworks

FastAPI

High-performance async APIs — industry standard for AI services

Flask

Lightweight and flexible — great for prototypes and microservices

Django

Full-featured framework for larger, data-driven AI applications

PYTHON WORKFLOW

Python in an AI Backend System

When a user submits a query to an AI application, the Python backend executes a precise sequence of operations — from constructing the request to delivering the final output. This is the loop you will write hundreds of times throughout this course.

Receive User Input

A user submits a prompt or request from the frontend application.

Construct API Request

The Python backend prepares the request payload — model parameters, prompt, and API key.

Send HTTP Request

The backend sends the request to an external AI service via an API call.

Receive API Response

The AI provider processes the request and returns a structured response.

Parse JSON Response

The backend extracts the useful content from the structured JSON response.

Return Result to User

The processed result is sent back to the frontend or calling application.

```
from openai import OpenAI

client = OpenAI()

response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content": user_input}]
)

result = response.choices[0].message.content
print(result)
```

This cycle repeats for every AI request your backend handles.

PACKAGE MANAGEMENT

Python Package Management with pip

What is pip?

`pip` is Python's built-in package installer. It connects to **PyPI (Python Package Index)** — the central repository of over 500,000 open-source Python packages — and downloads them into your environment.

Packages extend Python's core functionality, providing pre-built tools so you don't have to build everything from scratch.

Essential AI Engineering Packages

- `openai` — official OpenAI Python SDK
- `python-dotenv` — load environment variables from `.env` files
- `langchain` — LLM orchestration framework
- `fastapi` — high-performance API server

Installing Packages

```
# Install a single package
pip install openai

# Install multiple packages
pip install openai python-dotenv

# Install a specific version
pip install openai==1.3.5

# Install from requirements file
pip install -r requirements.txt
```

Always install packages inside a virtual environment — never into your global Python installation.

PROBLEM

The Dependency Conflict Problem

As you work on multiple projects, you will inevitably run into **dependency conflicts** - situations where different projects require incompatible versions of the same package. Without isolation, installing one project's dependencies can silently break another.

Project A

Requires `openai==0.28`
Legacy chatbot application

Conflict

Only one version can be installed globally at a time

Project B

Requires `openai==1.3.5`
New RAG pipeline

The solution is to give each project its own **isolated Python environment** with its own set of packages — this is exactly what virtual environments provide.

VIRTUAL ENVIRONMENTS

Creating Isolated Python Environments

What is a Virtual Environment?

A virtual environment is a **self-contained directory** that contains its own Python interpreter and a dedicated set of installed packages - completely isolated from your global Python installation and from other projects.

Why It Matters

- Eliminates dependency conflicts between projects
- Creates reproducible, shareable environments
- Keeps your global Python installation clean
- Industry-standard practice in every Python project

Creating a Virtual Environment

```
# Create a virtual environment named 'venv'
python -m venv venv

# What gets created:
venv/
├── bin/        # Python interpreter + scripts
├── lib/        # Installed packages live here
└── pyvenv.cfg  # Environment configuration
```

Always create a virtual environment at the root of every new project before installing any packages.

VIRTUAL ENVIRONMENTS

Activating Your Virtual Environment

Creating a virtual environment is not enough - you must **activate** it to direct Python and pip commands into that isolated environment. You will know it is active when you see `(venv)` prefixed in your terminal prompt.

macOS / Linux

```
# Activate
source venv/bin/activate

# Your prompt changes to:
(venv) $

# Deactivate when done
deactivate
```

Windows (Command Prompt)

```
# Activate
venv\Scripts\activate

# Your prompt changes to:
(venv) C:\project>

# Deactivate when done
deactivate
```

Windows (PowerShell)

```
# Activate
venv\Scripts\Activate.ps1

# Your prompt changes to:
(venv) PS C:\project>

# Deactivate when done
deactivate
```

Always activate your virtual environment before running `pip install` or `python` commands in a project.

DEPENDENCY MANAGEMENT

Reproducible Environments with requirements.txt

The Problem

Your virtual environment exists only on your machine. When a teammate clones your project — or when you deploy to a server — there are no packages installed. You need a way to **capture and share** your exact dependency list.

The Solution: requirements.txt

`requirements.txt` is a plain-text file listing every package and its exact version. Anyone can recreate your exact environment with a single command.

```
# Capture current environment
pip freeze > requirements.txt

# Recreate on another machine
pip install -r requirements.txt
```

Example requirements.txt

```
openai==1.3.5
python-dotenv==1.0.0
fastapi==0.104.1
uvicorn==0.24.0
langchain==0.0.340
pydantic==2.5.0
httpx==0.25.2
```

Best Practices

- Always commit `requirements.txt` to version control
- Pin exact versions for production stability
- Update periodically and re-test
- Never commit the `venv/` folder itself

PROBLEM → SOLUTION

Why Version Control Exists

Before version control, developers managed code changes manually — creating endless copies of files with names like `final`, `final_v2`, and `final_ACTUAL_FINAL`. This approach fails the moment you collaborate with others or need to trace a bug back in time.

Without Version Control

```
project_final.py
project_final_v2.py
project_final_latest.py
project_final_REAL.py
project_backup_tuesday.py
```

With Git

```
main.py

# Full history tracked:
git log --oneline
a3f2c1 Add error handling
b8e4d2 Integrate OpenAI API
c1a9f5 Initial commit
```

Git is a distributed version control system that tracks every change to your codebase, enables safe collaboration, and lets you roll back to any previous state instantly.

VERSION CONTROL

Git & GitHub -Two Different Things

Git — Local Version Control

Git is a **tool installed on your machine** that tracks changes to files over time. It maintains a complete history of your project in a hidden `.git` folder. You use Git entirely offline - no internet required.

- Tracks every line change with a timestamp and author
- Enables branching - work on features without breaking main code
- Allows rollback to any previous commit
- Merges contributions from multiple developers

GitHub - Remote Collaboration Platform

GitHub is a **cloud-based hosting platform** for Git repositories. It stores your code remotely and adds collaboration features on top of Git.

- Hosts repositories publicly or privately in the cloud
- Enables team collaboration via pull requests and code review
- Powers CI/CD pipelines and automated deployments
- Industry standard — every AI engineer uses it daily

Analogy: Git is Microsoft Word's track-changes feature. GitHub is Google Drive — where you store and share those documents.

GIT ESSENTIALS

Core Git Commands You Will Use Daily

These four commands form the foundation of your Git workflow. You will execute this sequence every single day as an AI engineer — clone a repo, make changes, stage them, commit with a message, and push to GitHub.

1

git clone

```
git clone https://github.com/user/repo.git
```

Download a remote repository to your local machine. Your starting point for any existing project.

2

git add

```
git add .        # stage all changes
git add main.py    # stage specific file
```

Stage changes — tell Git which modified files to include in the next commit.

3

git commit

```
git commit -m "Add OpenAI API integration"
```

Save a permanent snapshot of staged changes with a descriptive message.

4

git push

```
git push origin main
```

Upload your local commits to the remote GitHub repository for teammates to access.

SECURITY CRITICAL

Secure API Credential Management

API keys are sensitive credentials — treat them like passwords. Hardcoding them into your source code and pushing to GitHub is one of the most common and costly mistakes in AI engineering. Exposed keys lead to **unauthorised usage, data breaches, and significant financial charges**.

❌ Never Do This

```
# DANGEROUS — never commit API keys
import openai

api_key = "sk-proj-xxxxxxxxxxxx"
openai.api_key = api_key
```

If pushed to a public GitHub repo, bots scan and steal this key within **seconds**.

✅ Correct Approach — .env File

```
# .env file (never commit this)
OPENAI_API_KEY=your_key_here

# .gitignore (always include)
.env
venv/
__pycache__/

# main.py — load safely
from dotenv import load_dotenv
import os
load_dotenv()
api_key = os.getenv("OPENAI_API_KEY")
```

Rule: If it's a secret, it lives in `.env`. If it's in `.env`, it's in `.gitignore`. No exceptions.

SESSION 1 · WRAP-UP

Session Summary & What's Next

Already Covered

- **AI Backend Architecture** — the 5-layer stack from user to model
- **What APIs Are** — contracts for software communication
- **Python's Role** — middleware, frameworks, and the AI ecosystem
- **Virtual Environments** — isolated, reproducible project spaces
- **Git & GitHub** — version control and collaboration fundamentals
- **Secure Credentials** — `.env` files, `.gitignore`, and `python-dotenv`

🚀 Session 2 Preview

APIs in Practice

Make your first real API call to an AI model

JSON Payload Engineering

Structure requests and parse responses correctly

Calling AI Models

Live coding — integrate OpenAI API with Python

Error Handling

Build robust, production-grade API clients